

High-throughput Point-Cloud Accelerator with Sparsity-aware Hierarchical Neighbor Voxel Search and Skipping

Yun-Chia Yu[†], Suraj PN Reddy^{§†}, Aryan Devrani^{§†}, Anirudh Srinivasan[†], Saianudeep Reddy Nayini[†],
Sohyeon Kim[†], Sung-Joon Jang^{*}, Sang-Seol Lee^{*}, Mingu Kang[†]

[†]University of California, San Diego (UCSD), ^{*}Korea Electronics Technology Institute (KETI)
{yunchia, surajpnreddy, adevrani, asrinivasan, snayini, sohyeon, mingy}@ucsd.edu,
{sjjang0626, sslee81}@keti.re.kr

Abstract—Point cloud-based 3D sparse convolution networks are widely employed to process voxel features efficiently. However, the irregularity of voxel sparsity poses significant challenges, leading to increased hardware complexity and inefficiencies. We propose an algorithm-hardware co-design for sparse 3D convolution. At the algorithm level, an on-the-fly threshold-based voxel skipping is adopted, enhancing efficiency. At the hardware level, a hierarchical 3-stage Voxel Search and Skipping is developed to systematically narrow down the non-zero search space, enhancing both performance and hardware utilization. We implemented the proposed accelerator in a 65 nm process to demonstrate a 77.7% reduction in delay compared to a baseline design, which does not support the proposed sparsity adaptations. The proposed system also achieved the 1.34 $\times$ and 2.22 $\times$ higher energy efficiency and throughput as compared to the state-of-arts.

Index Terms—point-cloud, 3D object detection, accelerator, autonomous driving.

I. INTRODUCTION

Autonomous driving systems use advanced machine perception to extract insights from sensor inputs, enabling recognition and tracking of road users. LiDAR-generated point cloud (PC) data [1] is crucial for 3D object detection, providing spatial information to understand the environment [2] [3]. Perception tasks typically operate at up to 100 frames per second for timely hazard detection. However, PC-based 3D object detection models such as VoxelNet [4] face challenges in real-time and low-power execution, especially in resource-limited setups, due to the extensive processing required for 3D point data.

Recently, the high sparsity of PC data (often exceeding 97%) has been leveraged to skip redundant computations, enhancing processing speed [5]. However, the irregular distribution of sparse data points complicates hardware implementation, particularly in identifying active inputs and corresponding weights. Furthermore, in conventional convolutional operations, non-zero elements increase due to dilation, reducing sparsity benefits. Sub-manifold convolutions [6] address this by producing non-zero outputs only when the receptive field's center point is non-zero, maintaining sparsity. However,

this introduces additional hardware complexity in determining which inputs and weights to process or skip in real-time.

Previous works SECOND [7] stored all possible combinations of [input index–weight index–output index] in a hash table to facilitate the skipping of sparse pixels at the cost of brute-force large table and searching overhead. In contrast, SPADE [8] introduced a more advanced computing logic that eliminated the need for large hash table storage, thereby reducing memory overhead and improving area efficiency, albeit at the cost of increased computational complexity. However, SPADE focuses primarily on conventional 3D convolutions without discussing optimizations for sub-manifold convolutions.

We aim to address these challenges through a hardware and software co-optimization approach that employs dynamic and hierarchical voxel search and skipping mechanisms. This approach skips less critical voxel processing by evaluating their importance in real-time, all while maintaining minimal hardware overhead. Although this paper primarily focuses on 3D voxel-based models, the proposed hardware design is also compatible to pillar-based sparse approaches, demonstrating its versatility. The detailed contributions are as follows.

We propose a hierarchical voxel search mechanism that operates in three stages: (1) it initially identifies non-zero center voxels, and (2) subsequently searches for non-zero neighboring voxels within the receptive field surrounding the center voxel. These steps significantly reduce computation by focusing solely on non-zero center and neighboring voxels. However, brute-force searching can result in processing inefficiency, creating cycle bubbles when the searched coordinates are turned out to be zero. To address this, (3) we narrow down the search space by independently checking the x-, y-, and z-coordinates, further optimizing the process.

Building on the above hardware mechanism, we address the decrease in sparsity over layers due to the dilation by leveraging magnitude-based voxel skipping algorithm [9]. Although this approach does not reduce the overall count of active (non-zero) voxels, it identifies and selects key voxels essential for processing. To leverage this mechanism, we enhance the hardware with the capability to dynamically compute key

[§]Equal contribution

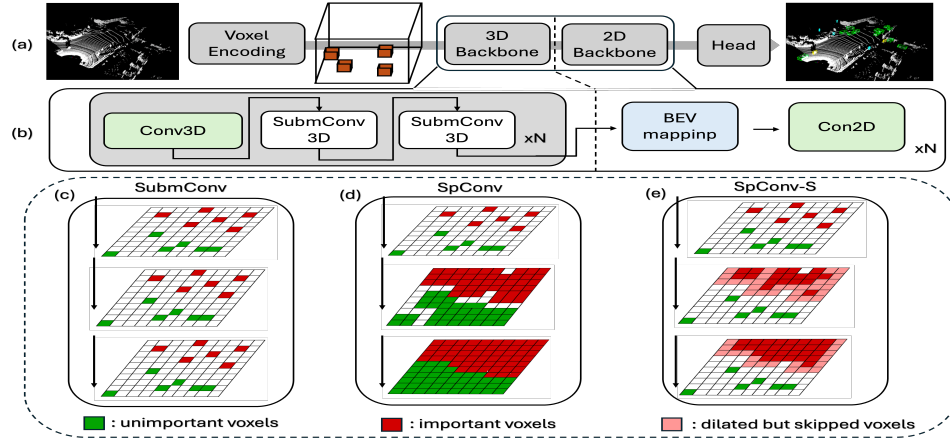


Fig. 1: Overview of voxel-based model: (a) model architecture, (b) Building blocks of 3D backbone. Comparison of output voxel set of various 3D sparse convolutions: (c) SubmConv, (d) SpConv, and (e) SpConv-S.

voxels on the fly during GEMM operations. These selected key voxels are then processed in subsequent layers, allowing the system to skip a significant portion of voxels and achieve a skipping ratio of approximately 65.7%.

To evaluate the efficacy of the proposed hardware and algorithm co-optimizations, we designed the accelerator using a 65 nm CMOS process and evaluated it with the Voxel R-CNN [10] architecture on the KITTI [11] dataset. Post-layout simulations indicate that the proposed architecture achieves 77.7% delay reductions.

II. BACKGROUND

This section provides an overview of voxel-based models and their key computational kernels. Additionally, we discuss a recently introduced method aimed at minimizing the computational costs of these models.

A. Voxel-based Model

Voxel-based 3D object detection involves representing a point cloud (PC) in 3D space by discretizing it into a regular voxel grid and aggregating the point cloud information within each voxel to form voxel features, enabling efficient processing with 3D convolutional neural networks (3D-CNNs). Each voxel is defined as $V = \{v_{p_k}\} = \{(p_k, f_{p_k})\}$, where $p_k = (x_k, y_k, z_k)$ represents the 3D coordinates of the k -th voxel, and f_{p_k} denotes its aggregated feature vector. This voxelization facilitates the extraction of spatial features for the 3D object detection. It is noteworthy that voxel-based models exhibit significant sparsity in raw data ($>98\%$), resulting in numerous inactive (all-zero) voxel positions due to the inherently sparse nature of 3D space. This paper aims to effectively exploit this opportunity to enhance computational efficiency.

B. Regular & Sub-manifold Sparse Convolution

Regular sparse convolution (SpConv) and sub-manifold convolution (SubmConv) are key kernels widely adopted in voxel-based models for 3D object detection. SpConv aggregates features within the receptive field of each input voxel, updating

both existing voxels and activating new ones. Formally, the output feature of voxel is computed as:

$$f_p^{out} = \sum_{k \in \mathcal{K}} w_k \cdot f_{p+k}^{in}, \quad (1)$$

where f_p^{out} and f_{p+k}^{in} are the feature of output voxel v_p^{out} and neighboring input voxel v_{p+k} , respectively. w_k denotes the kernel weights, and $\mathcal{K}$ represents the set of kernel offsets. The output voxel set for SpConv is defined as:

$$\mathcal{P}_{out} = \bigcup_{p \in \mathcal{P}_{in}} (p + \mathcal{K}), \quad (2)$$

where $\mathcal{P}_{in}$ ($\mathcal{P}_{out}$) is the set of active input (output) voxels' coordinates. Here, for simplicity, the stride of "1" is assumed. This expands the receptive field by activating new voxels as shown in Fig. 1d, improving feature propagation but increasing computational cost as layer goes by. SubmConv addresses the computing cost increase issue in SpConv by restricting the output voxel set to the original input voxel set:

$$\mathcal{P}_{out} = \mathcal{P}_{in}, \quad (3)$$

Here, the SubmConv generates a non-zero output only when the center of the receptive field is non-zero (Fig. 1c). By doing so, no additional voxels are activated, and sparsity is preserved before and after this kernel. While SubmConv is employed to reduce computation cost, it typically results in degraded accuracy. To address this trade-off, many voxel-based models [10] often alternate between SubmConv and standard kernels, as illustrated in Fig. 1, to balance accuracy and computational efficiency.

Additionally, the nature of SubmConv, which produces output only when the center of the receptive field is non-zero, offers an opportunity to skip more computations. However, this also increases hardware complexity, as it necessitates checking the center position before performing the MAC operation for any position between the kernel and input voxel. This paper will further leverage this opportunity by proposing the low-complexity sparse-aware processor.

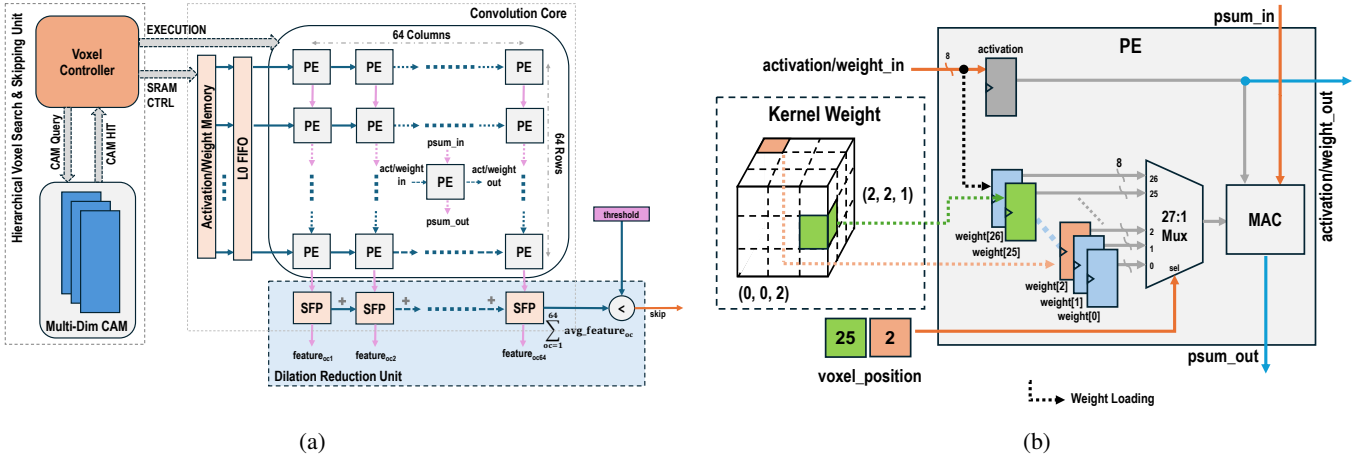


Fig. 2: (a) System architecture & data path illustrating convolution core and the Hierarchical Voxel Search & Skipping Unit (HVSS), (b) PE design illustrating weight stationary flow with all kernel weights stored to support MAC for active voxels.

C. Magnitude-based Dynamic Voxel Skipping

Magnitude-based dynamic voxel skipping [9] enhances the efficiency of sparse convolution by introducing a mechanism to skip less important voxels, significantly reducing the computational cost. The importance score of each voxel is computed by the sigmoid of the mean of its features. Based on a pre-defined percentile threshold T_p , the input voxel set is divided into two subsets: the important voxel set $\mathcal{P}_{im}$ and the unimportant voxel set $\mathcal{P}_{nim}$. Formally:

$$\mathcal{P}_{im} = \{p \in \mathcal{P}_{in} \mid S_p(f_p) \geq T_p\}, \quad \mathcal{P}_{nim} = \mathcal{P}_{in} \setminus \mathcal{P}_{im}, \quad (4)$$

where $S_p(f_p)$ is the importance score of voxel v_p in terms of percentile, e.g., high percentile with higher score. For SpConv with voxel skipping (SpConv-S), the SpConv is performed for the $\mathcal{P}_{im}$ whereas SubmConv is performed on $\mathcal{P}_{nim}$ (Fig. 2e). The output voxel set includes all unimportant voxels and the neighborhoods of important voxels, defined as:

$$\mathcal{P}_{out} = \mathcal{P}_{nim} \cup \bigcup_{p \in \mathcal{P}_{im}} (p + \mathcal{K}), \quad (5)$$

This indicates that the only $\mathcal{P}_{im}$ creates dilation while $\mathcal{P}_{nim}$ maintains its original receptive field. This method optimally balances accuracy and computational costs based on their importance. This paper further explores how to implement the above algorithmic innovation with low-cost hardware support.

III. PROPOSED SYSTEM

This section introduces hardware and software co-optimization approach that employs dynamic and hierarchical voxel search and skipping mechanisms to boost the speed and energy efficiency.

A. Architecture Overview

The proposed architecture includes 1) *convolution core*, which performs GEMM operations and 2) *Hierarchical Voxel Search & Skipping (HVSS)* Unit, which provides the sparse-adaptive computing capability to boost the performance. The

convolution core features a 2D systolic array optimized for 3D convolution as shown in Fig. 2a. While it is flexible, we choose 64×64 array size in this work. Each Processing Element (PE) performs Multiply-Accumulate (MAC) operations for 8-bit activation and weight data. Activation and weight data are loaded into on-chip SRAM, with an L0 FIFO managing retrieval during both the weight-loading into the PE array and execution stages. The architecture adopts a widely-adopted weight-stationary dataflow, where weights are stored in the register of PE as illustrated in Fig. 2b. In the proposed design, each PE has 27 registers to hold all the $3 \times 3 \times 3$ weight kernel elements for a specific input and output channel. Within the 64×64 PE array, activation data flows horizontally from the west to east, while partial sums (psums) move vertically down through each column. While the input channel index is mapped on the row index, each column represents an output channel, allowing concurrent processing across multiple channels and ensuring high computational throughput.

At the bottom of each column, a single row of Special Function Processors (SFPs) accumulates psums generated by the corresponding PE column. By delivering psums from non-zero voxel features to the SFP consecutively, intermediate psum storage in memory is eliminated. This enables the SFP to perform on-the-fly accumulation of psums and directly store the computed output feature in the output SRAM. Unlike the brute-force method, which requires 27 times of writing into psum memory for the computation of single output feature, our approach significantly reduces memory access by computing the output feature on the fly.

While this work focuses on the voxel-based approaches, the proposed hardware accelerator is also fully compatible with pillar-based models, offering flexibility for diverse point-cloud processing tasks. For pillar-based models, which rely on 2D convolution, each PE is configured to store nine weights, matching the kernel size of 3×3 . The remaining registers can be utilized to store the kernels for multiple channels to process in the time interleaved fashion.

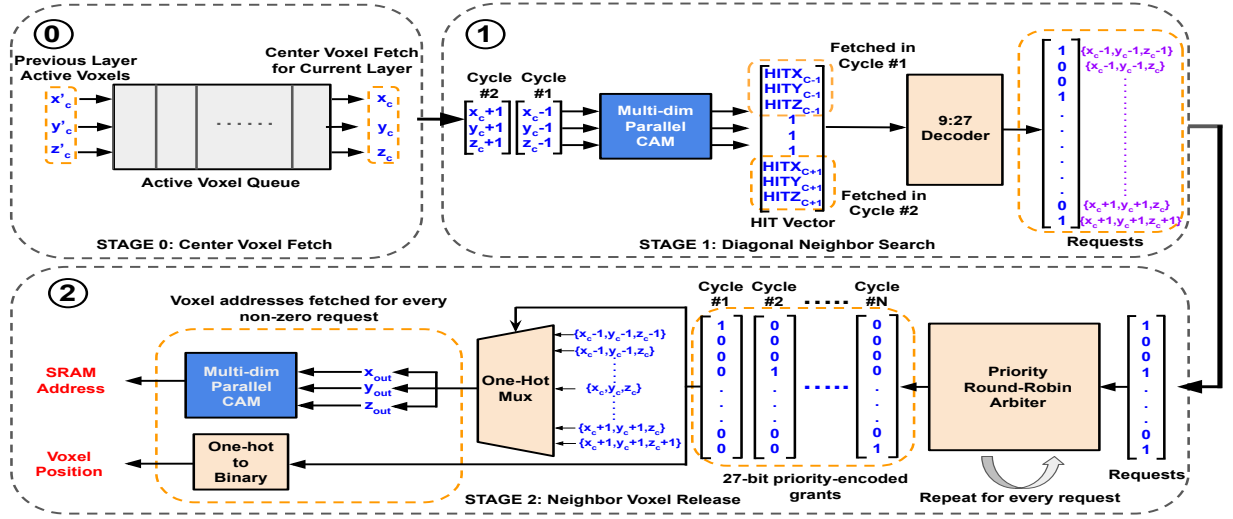


Fig. 3: 3-stage control flow for the Hierarchical Voxel Search & Skipping (HVSS) unit (all control signals are generated by the voxel controller, which is not shown for simplicity).

B. Hierarchical Voxel Search & Skipping (HVSS) Unit

The proposed architecture leverages the inherent sparsity in point cloud models, which often exhibits up to $> 98\%$. This is achieved through the HVSS unit, which integrates a voxel control unit, *Multi-Dim Parallel CAM*, and other auxiliary logic. The overall control flow of the unit can be broken down into the following key stages:

- **STAGE-0: Center Voxel Fetch:** For SubmConv, the output is generated only if the center voxel exists. Rather than brute-forcelly storing and computing zeroes for inactive voxel positions, we maintain an *Active Voxel Queue*, which is filled up during the computation of previous layer. This buffer queue contains the coordinates and physical memory addresses of active voxels. By doing so, only the active voxels are stored in the memory, saving the storage space dramatically. For computation, we bring the non-zero coordinates (x_c, y_c, z_c) directly from the active voxel queue to selectively compute for only non-zero center voxels, saving compute cycles significantly.
- **STAGE-1: Diagonal Neighbor Search:** Around the center voxel at (x_c, y_c, z_c) , the neighboring 26 voxels in 3D space need to be used for the computation to generate the output. However, due to the inherent high sparsity, many of these neighboring voxels remain inactive (zero). For example, on average, only about 50% of the neighboring voxels are active in the KITTI [11] dataset. This indicates a great opportunity to further reduce the energy and cycle count by leveraging this sparsity.

To exploit this, it is necessary to search for all the neighboring active voxels using CAMs, which requires a total of 26 cycles. However, due to high sparsity, around 50% of these searches will be a miss, indicating an inactive neighbor voxel, thereby wasting valuable search cycles. Such misses will create a worst-case of 26 bubbles in computing cycles for the convolution core, degrading the overall utilization.

To minimize these search bubble cycles, we propose a

Multi-Dim Parallel CAM for the x-, y-, z- parallel search. The neighbor search begins with obtaining two diagonally opposite corner coordinates - $(x_c - 1, y_c - 1, z_c - 1)$ & $(x_c + 1, y_c + 1, z_c + 1)$ by subtracting and adding "1" from the center coordinate obtained from STAGE-0. Then, $(x_c - 1)$ and $(x_c + 1)$ are searched from the CAM-X for two consecutive cycles (see Fig. 3 Stage-① and Fig. 4). This process confirms whether there is any active voxel whose x-coordinate is either $(x_c - 1)$ and $(x_c + 1)$. Note that we do not perform the search for x_c as the search result is guaranteed to match due to the presence of the center voxel. Similar operations are performed in parallel for CAM-Y and CAM-Z as well, enabling all three CAM operations to execute simultaneously. This diagonal neighbor search approach drastically reduces the number of search bubble cycles from a worst-case of 26 (where every neighbor is inactive) to merely two.

Through above operations, the 9-bit HIT vector ($\text{hit_x/y/z}[2:0]$) is generated by CAMs, which gets passed to the 9:27 decoder (see Fig.3 Stage-①), generating a 27-bit *Requests* vector. Here, each bit corresponds to one of the 27 coordinates in 3D space, with a "1" indicating a potentially active voxel in the corresponding position.

- **STAGE-2: Neighbor Voxel Release:**

Suppose the 27-bit *Requests* vector contains N "1"s. The round-robin arbiter will then generate N one-hot encoded 27-bit vectors in N consecutive cycles (see Fig. 3 Stage-②). The one-hot mux converts these 27-bit vectors (*Grants*) into x-, y-, z- coordinates which are used to fetch from the multi-dim parallel CAM. If the CAM indicates a complete "match", the voxel is now guaranteed to be active (non-zero). Once the active voxel neighbors are identified, each voxel's feature contents pointed by the output SRAM address, as well as the voxel position is passed to the convolution core for computation of the current active voxel with corresponding weight out of 27 registers (see Fig. 2b). By pipelining Stages-

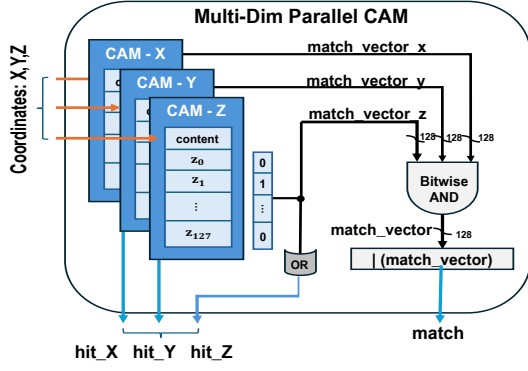


Fig. 4: Multi Dimension Parallel CAM - A wrapper integrating the three dedicated CAMs to store non-zero voxel coordinates.

① and ②, the search takes a total N cycles, which is significantly lower than 26 as will be shown in Section IV.

C. On-the-fly Dilation Reduction

While the original voxel skipping algorithm [9] introduced in Section II-C was designed to demarcate important and redundant voxels based on their percentile score among all voxels, it has a significant limitation: it does not allow for on-the-fly separation. Specifically, the decision about a voxel's importance cannot be made until all elements are processed, as sorting is required to rank them in magnitude order. To address this limitation, we modified the algorithm to be more hardware-friendly by using a comparison with an absolute-magnitude threshold (T) instead, as follows:

$$\mathcal{P}_{\text{im}} = \{p \in \mathcal{P}_{\text{in}} \mid S(f_p) \geq T\}, \quad \mathcal{P}_{\text{nim}} = \mathcal{P}_{\text{in}} \setminus \mathcal{P}_{\text{im}}, \quad (6)$$

where $S(f_p)$ is the absolute importance score computed as the sigmoid of the mean of its feature. The value of T is determined from the training data by maximizing it within the range where accuracy remains consistent.

To capitalize on the above opportunity, the proposed architecture incorporates a threshold-based voxel skipping mechanism that dynamically evaluates the importance of computed output features. This is achieved by leveraging the Special Function Processors (SFPs) within the systolic array, which compute the absolute mean of the output feature across all output channels.

As illustrated in Fig. 2a, the mean computation is achieved through the horizontal movement of averaged output feature across the SFPs. The final mean value is fetched at the output of the last SFP. Note that the sigmoid function is monotonic. Therefore, instead of implementing hardware to compute the sigmoid directly, we compare the computed average with the pre-stored $\text{sigmoid}^{-1}(T)$ using an on-chip comparator.

By performing the mean calculation during the convolution process itself, the architecture avoids the additional cost of fetching psums from memory and computing the mean separately, thereby saving cycles and enhancing overall efficiency. If the computed mean falls below the threshold, the corresponding output feature is skipped for the subsequent layer. This dynamic filtering approach mitigates the effects of

dilation, maintaining sparsity and improving the efficiency of subsequent processing stages.

IV. EVALUATIONS

A. Evaluation Setup

To evaluate the efficacy of the proposed techniques, including hierarchical neighbor search and on-the-fly dilation reduction, we select a representative voxel-based model, Voxel R-CNN [10], as the baseline. Furthermore, we apply the threshold-based voxel skipping algorithm to this model, denoted as Voxel R-CNN-S, to facilitate comparisons with the vanilla model. For this study, we utilize the KITTI [11] dataset as the benchmark. To access the performance of proposed hierarchical neighbor voxel searching (HierS), we choose brute-force neighbor voxel searching (BFS) as baseline, where each center voxel will perform searching to all of the voxels within its receptive window.

Our design, including the proposed Hierarchical Voxel Search and Skipping Unit (HVSS) and the systolic array-based convolution core, was implemented and evaluated using a comprehensive RTL-to-GDSII flow. The RTL design was synthesized and placed-and-routed (PnR) using a 65 nm CMOS process with a general-purpose (GP) standard cell library. The implementation achieved timing closure at a 1 GHz clock frequency. To model hardware behavior, a cycle-accurate simulator was developed to calculate cycle counts, read/write operations on the on-chip SRAM, and search operations in the CAM. To evaluate power consumption, we performed post-layout power analysis using a value change dump (VCD) file, which captures realistic toggling behavior based on simulated data flow patterns. For SRAM memory, we use a Memory Compiler with ARM High density 65 nm GP single-port SRAM version, while the 65 nm CAM search energy and speed are derived from [12]. Key performance metrics, including power, performance, & area (PPA), are analyzed and compared with existing state-of-the-art accelerators, highlighting the efficiency of our proposed architecture.

B. Evaluation Results

• **Threshold-based voxel skipping ratio analysis:** The layer-wise analysis of number of important voxels within the 3D backbone of Voxel R-CNN & Voxel RCNN-S is shown in Fig. 5a. Voxel R-CNN-S effectively skips 65.7% of the voxels on average while achieving a slight accuracy improvement of 0.48%, significantly improving computational efficiency.

• **Voxel neighbor search counts reduction analysis:** The Layer-wise analysis of average neighbor search counts reduction is shown in Fig. 5b. Compared to BFS, HierS achieves 5.16 & 6.34 search counts reduction in Voxel R-CNN & Voxel R-CNN-S, respectively, with minimal 0.71% area overhead and 0.014% power overhead.

• **Energy, area, and throughput analysis** The normalized energy efficiency and throughput analysis are illustrated in Fig. 5c. Compared to BFS, HierS achieves a 1.20 $\times$ and 1.24 $\times$ improvement in energy efficiency and throughput for Voxel R-CNN and Voxel R-CNN-S, respectively. Thanks to the

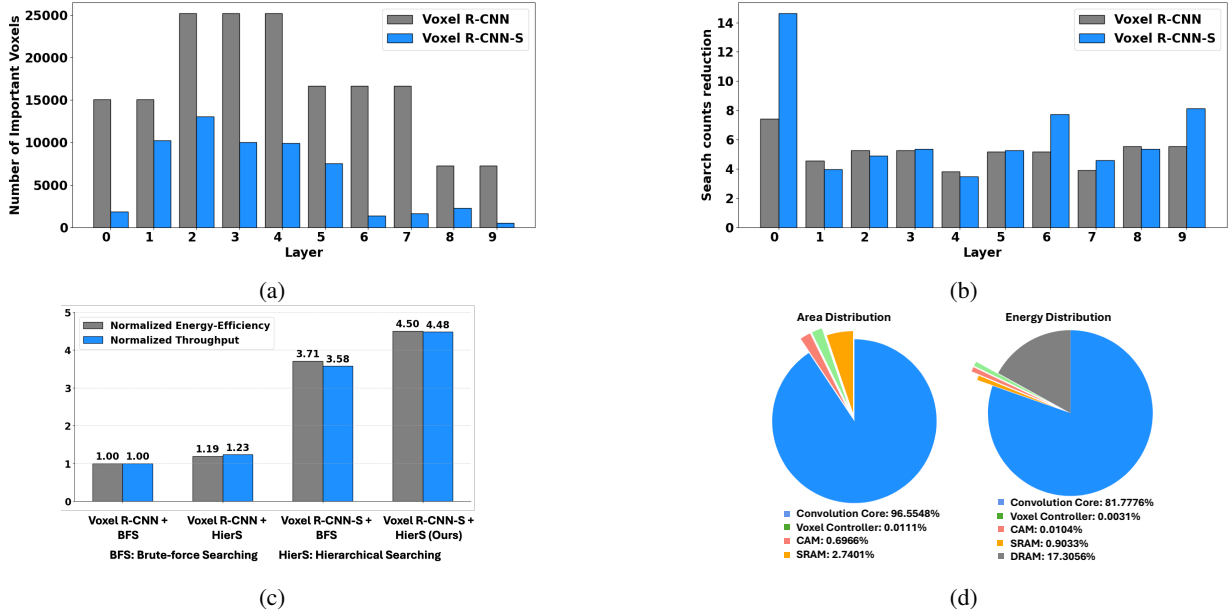


Fig. 5: (a) Layer-wise analysis of number of important voxels within the 3D backbone of Voxel R-CNN & Voxel R-CNN-S, (b) Layer-wise analysis of number of neighbor search counts reduction, (c) Comparison of normalized energy-efficiency & throughput, (d) Area & energy distribution of accelerator components.

TABLE I: Hardware Evaluation: Comparison with State-of-the-Art point-cloud accelerators.

	PointAcc	SPADE	Naive	Proposed (HW)	Proposed (HW+SW)
Cores	64x64=4096				
SRAM (KB)	776	654	156		
Area (mm^2)	15.70 (34.41*)	11.60 (38.28*)	45.28		
Frequency	1 GHz				
DRAM Bandwidth	HBM-2 256 GB/s				
Technology	40nm	32nm	65nm		
Peak Performance	8 TOPS				
Precision	N/A	A8-W8-Acc32			
GOPS/ mm^2	521.80 (132.08*)	706.2 (112.89*)	180.92	223.06	810.51
GOPS/W	N/A	1365.3 (299.29*)	336.1	400.3	1514

* After scaling to a 65 nm technology by the rule in [13].

proposed on-the-fly dilation reduction technique, our hardware can effectively translation algorithmic computation efficiency into tangible actual hardware improvements. By leveraging the synergy between HierS with the on-the-fly dilation reduction, an overall $4.48\times$ improvement is achieved compared with baseline setting. Fig. 5d breaks down the area and power consumption. It can be noticed that Voxel Controller and Multi-Dim CAM only covers less than 0.02% energy and 1% area overhead, respectively.

• **Hardware overhead:** Note that even with three separate CAMs in HVSS, there is no additional storage overhead, as each CAM stores exclusive content. Moreover, all the auxiliary

logic outside of the CAMs (depicted in yellow in Fig. 3) consists of highly simplified circuitry, completing operations within a single cycle at 1 GHz, including a single CAM search.

• **Comparison with State-of-the-art:** Table I shows the comparison of proposed work with SOTA hardware accelerators. When matched in technology scaling factor [13], our work demonstrates $1.34\times$ higher energy efficiency over SPADE, and $2.22\times$ higher throughput over PointAcc.

V. CONCLUSION

The sparsity in voxels presents a great opportunity for aggressive energy and delay efficiencies, yet the irregularity of voxel coordinates poses new challenges for the efficient processing of 3D Point Cloud perceptrons. Our hardware-algorithm co-optimized accelerator addresses these challenges by leveraging the inherent sparsity of voxels and proactively enhancing it through on-the-fly dilation reduction, enabling the skipping of additional computations. The proposed HVSS mechanism is seamlessly integrated to further reduce redundant computations, achieving high computational efficiency with minimal accuracy degradation and negligible energy and area overheads. This breakthrough facilitates real-time 3D object detection at the edge, empowering applications in autonomous driving, robotics, AR/VR, and beyond.

ACKNOWLEDGMENT

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.2022-0-00394, Development of AI semiconductor for level 4 self-driving based on functional safety) and by the PRISM center in JUMP 2.0, an SRC program sponsored by DARPA.

REFERENCES

- [1] Y. Li, L. Ma, Z. Zhong, F. Liu, M. A. Chapman, D. Cao, and J. Li, "Deep learning for lidar point clouds in autonomous driving: A review," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 8, pp. 3412–3432, 2021.
- [2] B. Li, "3d fully convolutional network for vehicle detection in point cloud," 2017. [Online]. Available: <https://arxiv.org/abs/1611.08069>
- [3] X. Chen, H. Ma, J. Wan, B. Li, and T. Xia, "Multi-view 3d object detection network for autonomous driving," 2017. [Online]. Available: <https://arxiv.org/abs/1611.07759>
- [4] Y. Zhou and O. Tuzel, "Voxelnet: End-to-end learning for point cloud based 3d object detection," 2017. [Online]. Available: <https://arxiv.org/abs/1711.06396>
- [5] H. Tang, Z. Liu, X. Li, Y. Lin, and S. Han, "Torchsparse: Efficient point cloud inference engine," 2022. [Online]. Available: <https://arxiv.org/abs/2204.10319>
- [6] B. Graham and L. van der Maaten, "Submanifold sparse convolutional networks," 2017. [Online]. Available: <https://arxiv.org/abs/1706.01307>
- [7] Y. Yan, Y. Mao, and B. Li, "Second: Sparsely embedded convolutional detection," *Sensors*, vol. 18, no. 10, 2018. [Online]. Available: <https://www.mdpi.com/1424-8220/18/10/3337>
- [8] M. Lee, S. Park, H. Kim, M. Yoon, J. Lee, J. W. Choi, N. S. Kim, M. Kang, and J. Choi, "Spade: Sparse pillar-based 3d object detection accelerator for autonomous driving," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 454–467.
- [9] J. Liu, Y. Chen, X. Ye, Z. Tian, X. Tan, and X. Qi, "Spatial pruned sparse convolution for efficient 3d object detection," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22. Red Hook, NY, USA: Curran Associates Inc., 2024.
- [10] J. Deng, S. Shi, P. Li, W. Zhou, Y. Zhang, and H. Li, "Voxel r-cnn: Towards high performance voxel-based 3d object detection," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 2, pp. 1201–1209, May 2021. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/16207>
- [11] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the kitti vision benchmark suite," in *2012 IEEE Conference on Computer Vision and Pattern Recognition*, 2012, pp. 3354–3361.
- [12] K. Lee, W. Choi, and J. Park, "A 65-nm 0.6-fj/bit/search ternary content addressable memory using an adaptive match-line discharge," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 8, pp. 2574–2584, 2021.
- [13] A. Stillmaker and B. Baas, "Scaling equations for the accurate prediction of cmos device performance from 180nm to 7nm," *Integration*, vol. 58, pp. 74–81, 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167926017300755>